

01. Claude Code 源码学习

本篇笔记由【三黄工作室】微信公众号提供。欢迎关注。

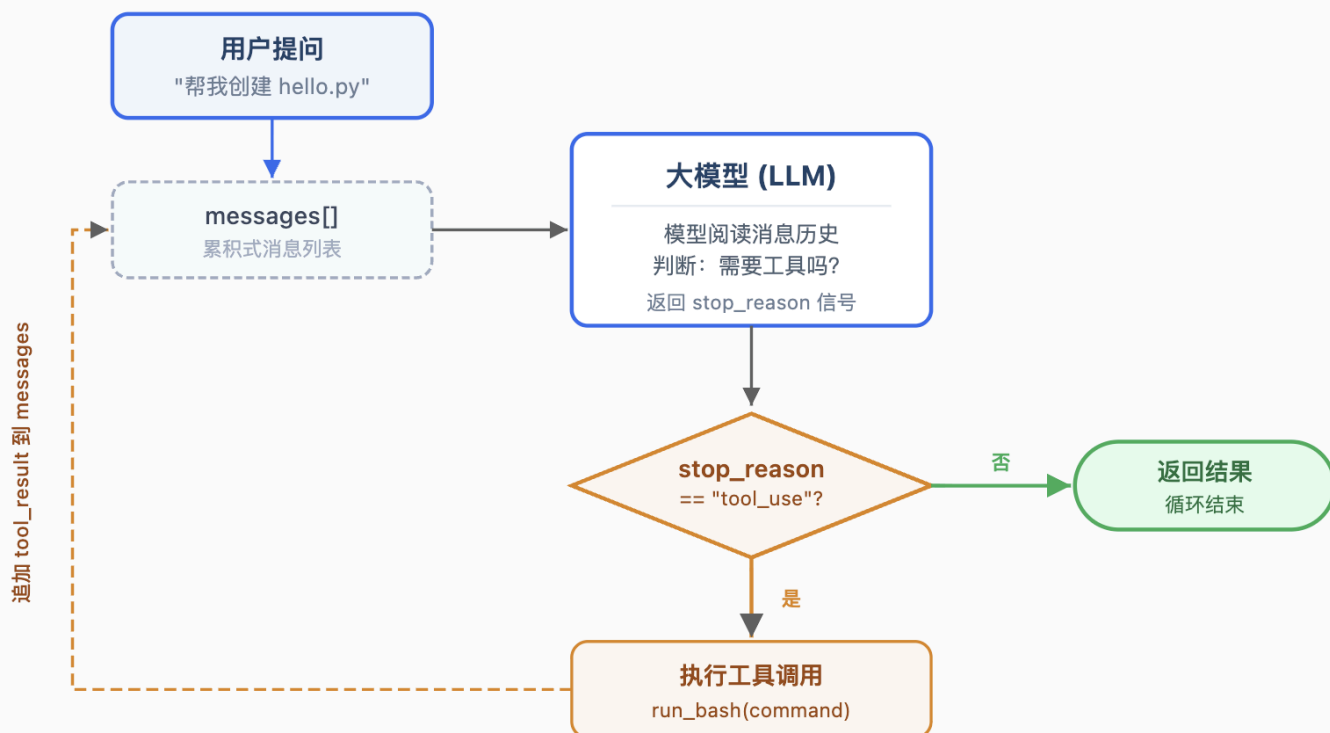


本文内容为 Claude Code 源码学习与 Learn Claude Code <https://learn.shareai.run/zh/> 的阅读笔记，会在教程的基础上，自主搭建出一个可运行的 Agent Loop 的基础架构。**一边完善 Agent 代码一边学习！**

1. 第一章 Agent Loop

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

Agent Loop — 一个 while 循环驱动整个 Agent



核心：一个 `while True` 循环。模型调工具 → 执行 → 喂回 → 再问。不调工具就停。
后续所有章节都在这个循环上叠加机制。

1.1 概述

学习 Agent 的第一步，先建立最小闭环，把 Loop 跑起来：先把用户输入放进 messages，由模型决定要不要调工具，结果再回写到同一条循环里。

1.2 设计决策

1. **Tools 并不是越多越好，这个问题 GA 的文档里面也有提到过。有些 Tool 存在功能重复，比如 read file 就有很多种命令都可以实现。**

Bash 能读写文件、运行任意程序、在进程间传递数据、管理文件系统。任何额外的工具（read_file、write_file 等）都只是 bash 已有能力的子集。

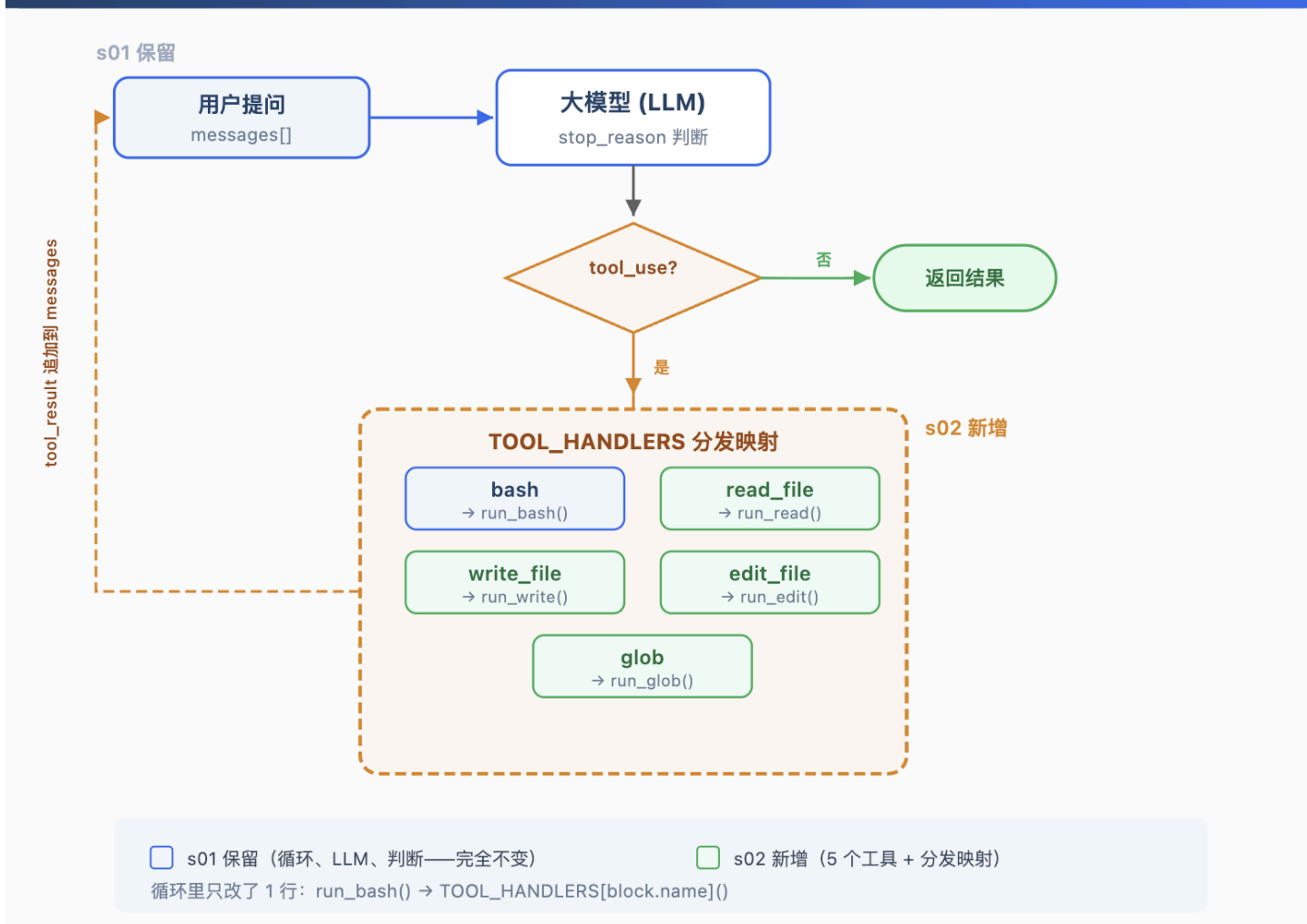
增加工具并不会解锁新能力，只会增加模型需要理解的接口。模型只需学习一个工具的 schema，实现代码不超过 100 行。这就是最小可行 agent：一个工具，一个循环。

2. **危险命令是通过在执行工具调用时，比如 Bash 命令，通过限制一些危险的命令关键词的执行来做到的。**
3. **生产版有多条退出和继续路径，覆盖 blocking limit、prompt too long、model error、abort、hook stop、max turns、token budget continuation、reactive compact retry 等场景。每种场景都有对应的恢复或退出策略。**

2. 第二章 Tool Use

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

Tool Use — 循环不变，只加分发映射



2.1 概述

在第一章的基础上，Tool Call 只有 1 个 Bash，Bash 在执行读写时会有很多意想不到的问题，比如特殊字符问题、转移、编码、空格等等问题。

理想的 Tool Call 设计，read_file 直接读取，无转义问题；write_file 内容原样写入，不经过 Shell 解释；edit_file 精确行号替换，可 diff 预览；bash 只负责执行命令。

因此，要增加 Tool，新增读、写、编辑。

2.2 设计决策

1. 为什么只设计 5 个工具？

(这个问题 GA 的文档里面介绍的更加详细) 这四个工具分别是 `bash`、`read_file`、`write_file`、`edit_file` 和 `glob`，覆盖了大约 95% 的编程任务。`Bash` 处理执行和任意命令；`read_file` 提供带行号的精确文件读取；`write_file` 创建或覆盖文件；`edit_file` 做精确的字符串替换。

工具越多，模型的认知负担越重——它必须在更多选项中做选择，选错的概率也随之增加。更少的工具也意味着更少的 schema 需要维护、更少的边界情况需要处理。

2. 工具结果持久化

每个工具有一个 `maxResultSizeChars` 字段。结果超过这个值就落盘，模型看到的是预览 + 文件路径。`FileRead` 特殊——设为 `Infinity`，防止读文件的输出又被当成文件落盘。具体来说，如果 `FileRead` 的结果超过阈值被落盘，模型下次读那个落盘文件时又会触发落盘 → 无限循环 (读文件 → 落盘 → 再读 → 再落盘 → ...)。

3. 工具调用的并发控制

工具存在只读、可并发等问题，当模型输出多个工具调用的 schema 后，要对这些工具调用分成若干个 batch，分批执行，提高效率。

2.3 工程化落地时的难点

当模型一口气发出多个工具调用时，系统到底按什么规则执行、并发、回写、合并上下文？

系统一旦变得庞大和复杂，棘手的问题会变成下面这些：

- 多个工具能不能并行执行
- 哪些工具必须串行
- 工具执行过程中要不要先发进度消息
- 并发工具的结果应该按完成顺序回写，还是按原始出现顺序回写
- 工具执行会不会改共享上下文
- 多个并发工具如果都要改上下文，最后怎么合并
- 工具调用失败后如何降级

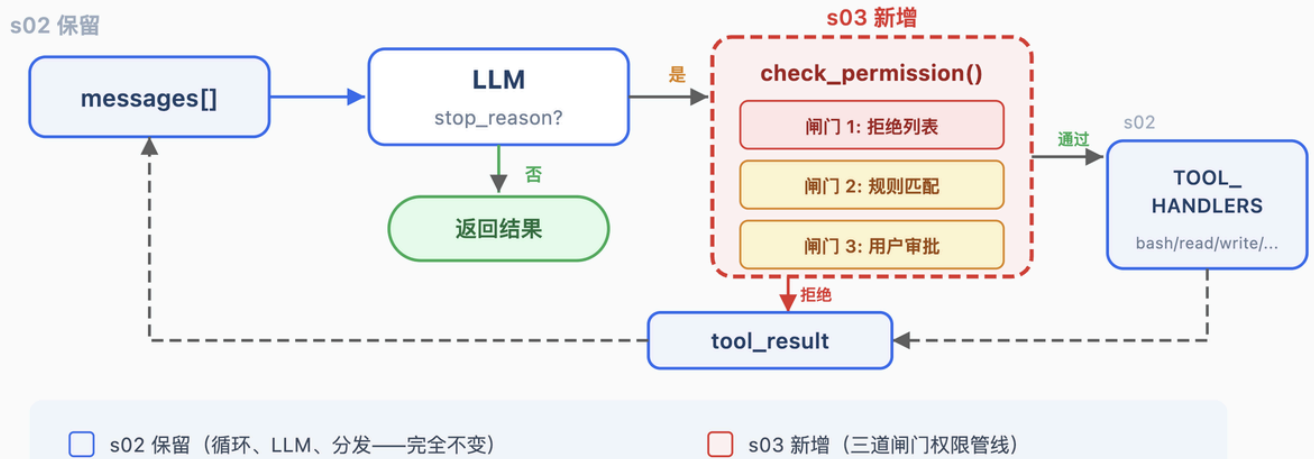
这些问题已经不是“工具注册”能解释的了。

 目前如果仅仅是工具调用，让 AI 在几分钟就能写出一个原型代码，但是真正有技术含量的就是在不同的应用场景应用不同的工具调用策略。

3. 第三章 Permission

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

Permission — 循环不变，工具执行前加一道门



执行工具调用时，通过询问权限来决定是否操作。主要是针对 bash、write、update、read、mcp 等操作。

常见的几种权限模式有：

- Default:
- Plan
- Auto

plan（计划模式）— 最严格

- 所有 WRITE_TOOLS (bash、write_file、edit_file) 直接 deny
- 读操作 (read_file) 自动 allow
- 不会问用户，适合先规划方案再执行的场景

auto（自动模式）— 半自动

- READ_ONLY_TOOLS (read_file) 自动 allow
- 写操作跳过 deny 规则后，走 allow 规则匹配，没匹配到就 ask 用户
- 适合需要快速审批的场景

default（默认模式）— 全交互

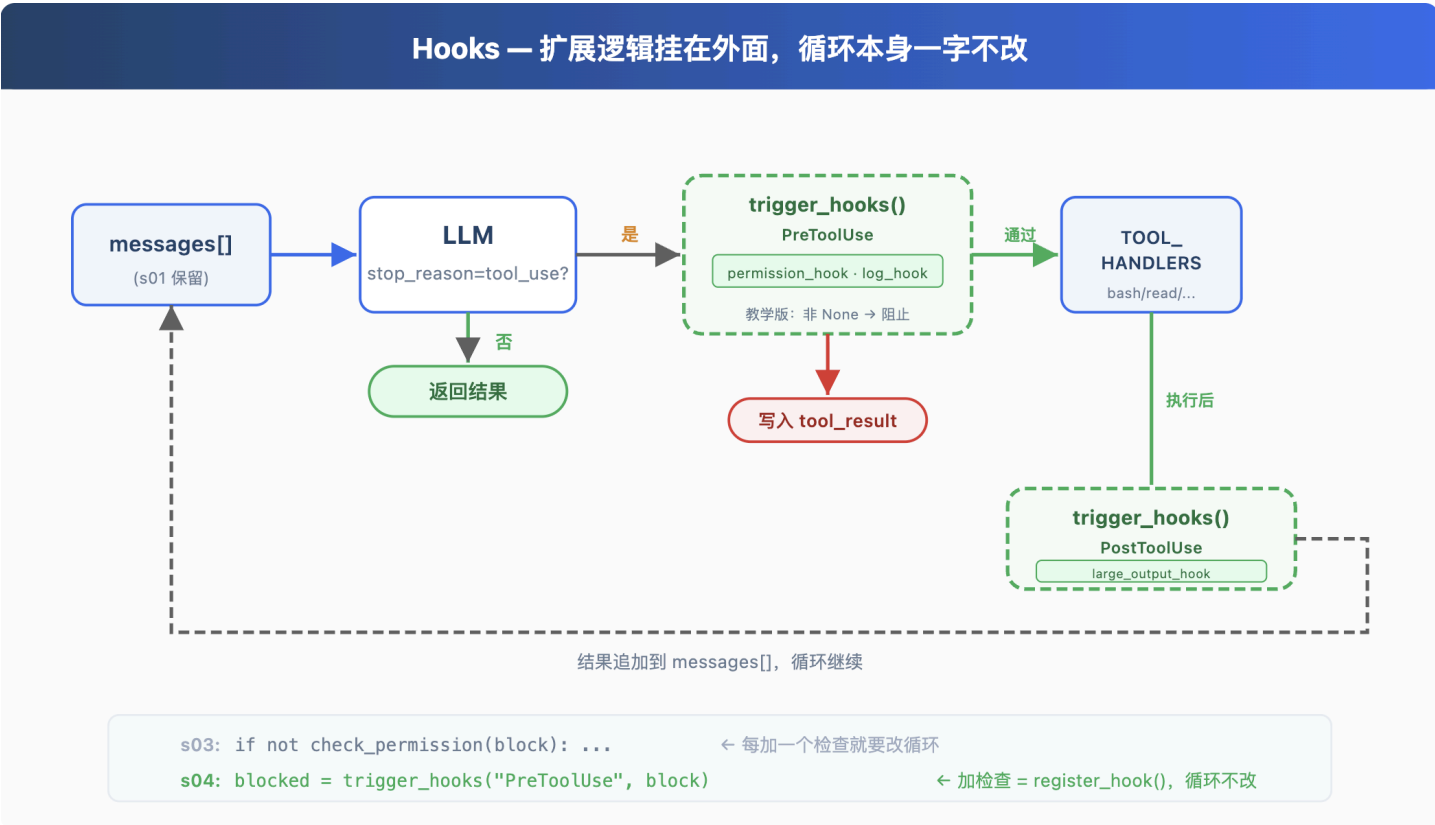
- 不做模式级拦截，完全走规则管道：deny 规则 → allow 规则 → ask 用户
- 每个未匹配规则的工具都会弹出交互式审批

三种模式都共享同一套 deny 规则（如 rm -rf /、sudo 直接拒绝），只是对未明确匹配的工具处理方式不同。

	plan	auto	default
read	allow	allow	规则/ask
write	deny	规则/ask	规则/ask
bash	deny	deny	deny

4. 第四章 Hooks

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>



4.1 概述

Hook（钩子） 是一种**非侵入式、可插拔**的扩展机制，允许你在 Agent 执行流程的**关键节点**（如思考前、调用工具前、返回结果前）插入自定义逻辑，**不修改核心代码**就能增强、监控、修改 Agent 行为。

简单说：**Agent 跑流程，Hook 截流、加工、控制、记录。**

其实也就是生命周期，这在前端里面非常常见，比如 beforeCreate、afterCreate 等等这些页面加载的节点钩子。

循环应该是一个稳定的核心，扩展应该挂在外面。

4.2 常见应用场景

1. 日志记录：在工具调用前后记录日志、统计时间
2. 内容过滤：过滤各个环节产生的内容中有的敏感信息
3. 权限控制：高危操作拦截
4. 上下文管理：可以在对话前后**自动加工记忆、上下文、系统提示词**。
5. 工具调用增强：在**工具调用前后**做加工，让工具更强大、更稳定。参数校验与格式化、结果加工与缓存。

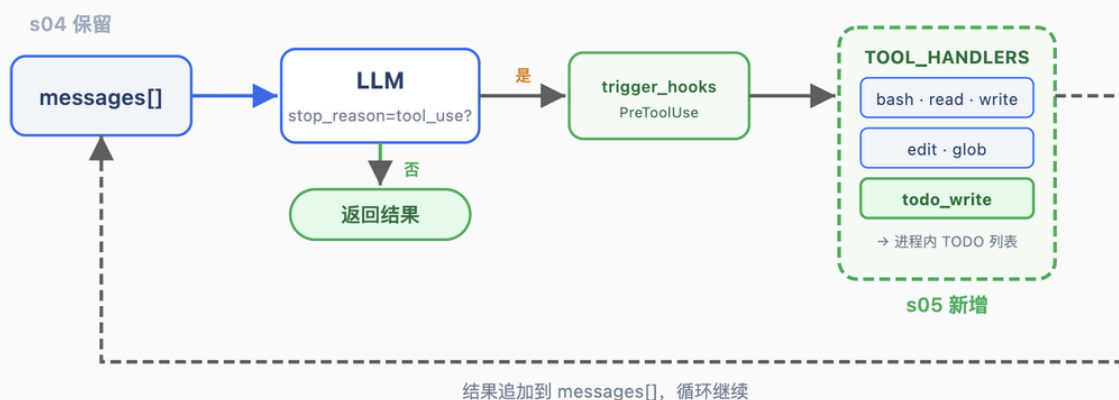
4.3 常见的节点

代码块

```
1  on_user_input      # 用户输入后
2  before_think       # 思考前
3  on_thought         # 思考中
4  before_tool_call   # 调用工具前
5  on_tool_result     # 工具返回后
6  before_llm_call    # 调用大模型前
7  after_llm_response # 大模型返回后
8  on_error           # 出错时
9  on_task_end        # 任务结束
```

5. 第五章 TodoWrite

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>



Nag Reminder (催更机制)

模型连续 3 轮没调 `todo_write` → 自动注入 `<reminder>Update your todos.</reminder>`

☐ s04 保留 (循环、钩子、5 个基础工具)

☐ s05 新增 (todo_write + nag reminder)

5.1 概述

当任务涉及多步操作时，容易走一步忘一步，明明已经做过的检查，会重复再做，一口气列出很多步骤后，很快又回到即兴发挥。这是因为模型虽然“能规划”，但它的当前注意力始终受上下文影响。

如果没有一块显式、稳定、可反复更新的计划状态，大任务就很容易漂。

所以在这一章要补上的，不是“更强的工具”，而是让 agent 把当前会话里的计划外显出来，并且持续更新。

5.2 设计决策



会话内 `todo` 不是持久化记忆，也不是后台任务管理，而是会话内部的轻量计划，帮助模型做任务推进的追踪。



通过 `TodoWrite` 让计划对 AI 可见。

我们不让模型在思维链中默默规划，而是强制通过 `TodoWrite` 工具将计划外化。每个计划项都有可追踪的状态 (`pending`、`in_progress`、`completed`)。这有三个好处：

1. 用户可以在执行前看到 agent 打算做什么；
2. 开发者可以通过检查计划状态来调试 agent 行为；

3. agent 自身可以在后续轮次中引用计划，即使早期上下文已经滚出窗口。



同一时间只允许一个任务进行中

TodoWrite 工具强制要求任何时候最多只能有一个任务处于 in_progress 状态。如果模型想开始第二个任务，必须先完成或放弃当前任务。这个约束防止了一种隐蔽的失败模式：试图通过交替处理多个项目来'多任务'的模型，往往会丢失状态并产出半成品。顺序执行的专注度远高于并行切换。



计划项上限为 20 条

TodoWrite 将计划项限制在 20 条以内。这是对过度规划的刻意约束。不加限制时，模型倾向于将任务分解成越来越细粒度的步骤，产出 50 条的计划，每一步都微不足道。冗长的计划很脆弱：如果第 15 步失败，剩下的 35 步可能全部作废。20 条以内的短计划保持在正确的抽象层级，更容易在现实偏离计划时做出调整。

5.3 实际应用中的体会

我在实际使用中，发现 Agent 如果加了 Plan，那么对于同一种事情（比如生成特定的业务报告），即使写了很多 System Prompt 和 Skills，模型每次都倾向于做一样的 Plan，导致输出同质化。

Agent 领域里，这叫：Plan Collapse（计划坍缩）。

因为当前大多数 Agent 的 Plan 本质是：“显式 CoT（Chain of Thought）的结构化模板化”。

深层原因是 LLM 会主动“压缩策略空间”模型不是在“创造”。模型是在：找最稳定、最低熵、最高概率的推理路径。而固定 Plan 会让某条路径概率越来越高。

5.4 其他解决方案

Plan 不再是固定树，而是：动态漂移图（Dynamic Reasoning Graph）

例如：

代码块

```
1  observe
2  → think
3  → tool
4  → rethink
```

```
5 → branch
6 → rollback
7 → retry
8 → explore
```

而不是：

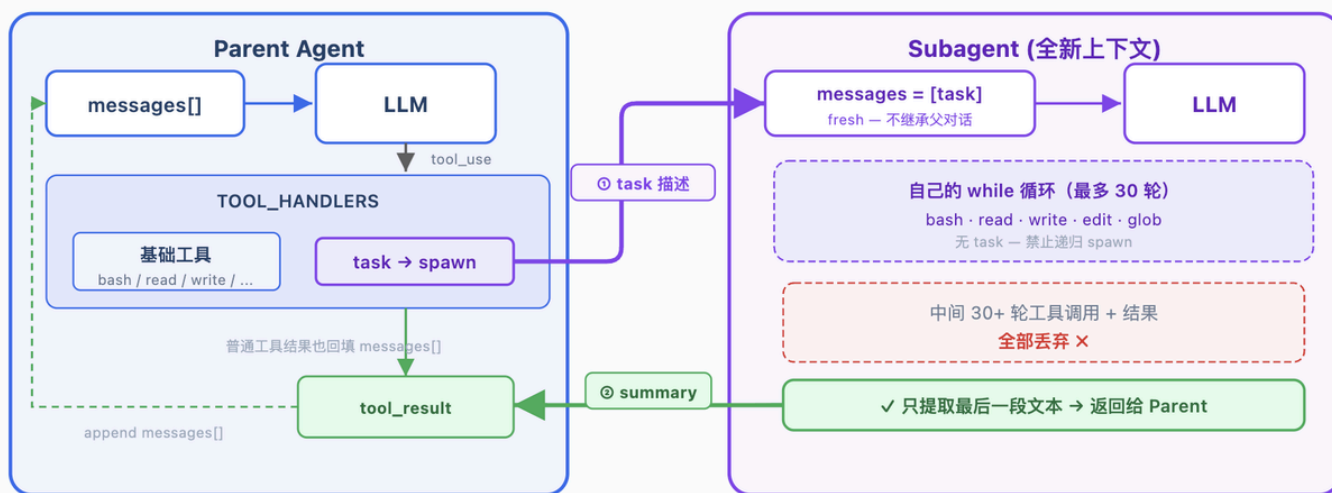
代码块

```
1 step1 → step2 → step3
```

6. 第六章 Subagent

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

Subagent — 独立 messages[], 中间过程全部丢弃



□ s05 保留：循环、hook、todo_write、6 个基础工具

□ s06 新增：task 工具 + spawn_subagent() — 独立 messages[], 只回传摘要

① Parent → Sub: task description (一小段文字)
② Sub → Parent: extract_text() (只有最终结论)

6.1 概述

SubAgent 解决的问题是节省上下文。对于某些只关系结果的步骤，理论上不需要把求取该结果的过程都注入到主 Agent 的上下文中来。

简而言之就是：把局部任务放进独立上下文里做，做完只把必要结果带回来。

6.2 设计决策

子智能体通常不需要拥有和父智能体完全一样的能力。

最小版本里，常见做法是：

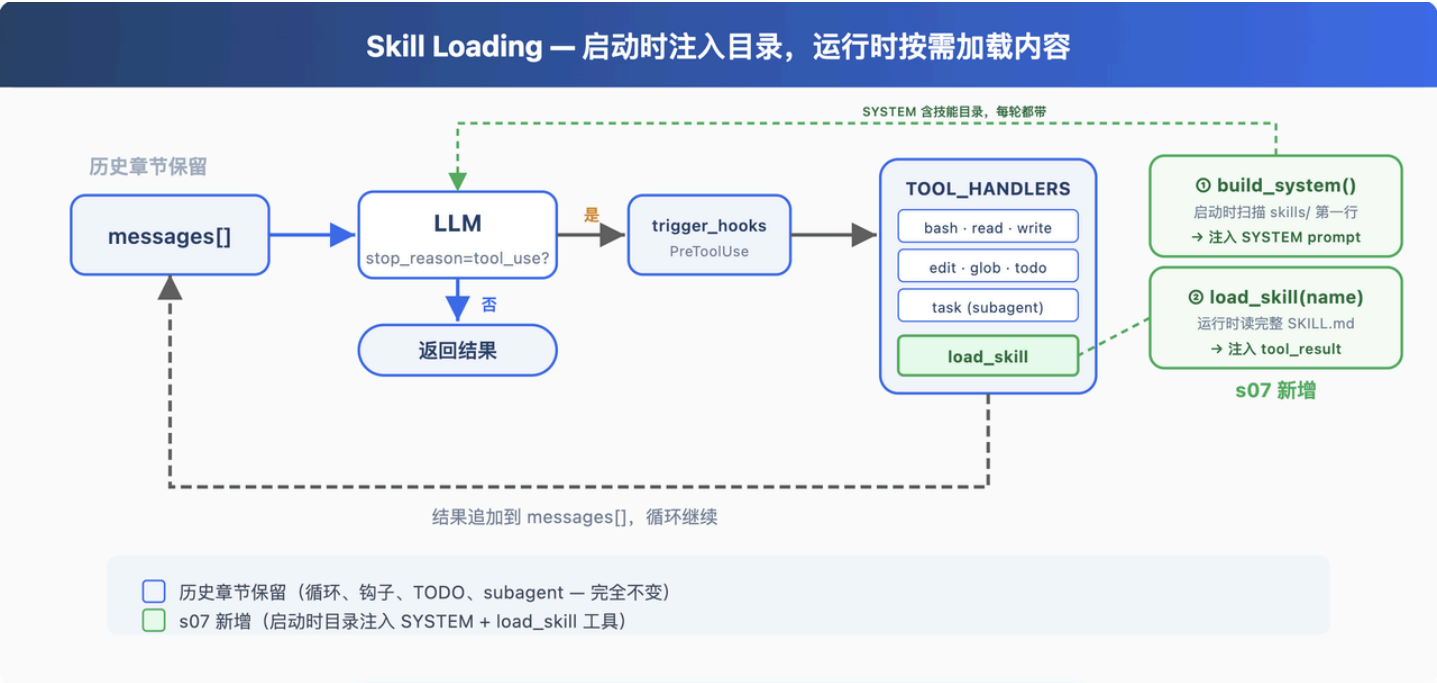
- 给它文件读取、搜索、bash 之类的基础工具
- 不给它继续派生子智能体的能力

这样可以防止它无限递归。

子智能体做完事后，不把全部内部历史写回去，而是返回一段总结。

7. 第七章 Skills

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>



Skills 的核心理念就是按需加载。

不同任务需要的领域知识不一样，如果把这些知识全部塞进 system prompt，就会出现两个问题：

1. 大部分 token 都浪费在当前用不到的说明上
2. prompt 越来越臃肿，主线规则越来越不清楚

所以 Skill 要把“长期可选知识”从 system prompt 主体里拆出来，改成按需加载。

常规的 skill 格式如下：

```
1  ---
2  name: code-review
3  description: Review code for bugs, performance issues, and style problems
4  ---
5
6  When reviewing code, check the following:
7
8  1. Correctness – logical errors, off-by-one, missing edge cases, null/None
   handling
9  2. Security – command injection, path traversal, XSS, SQL injection,
   hardcoded secrets
10 3. Performance – unnecessary allocations, N+1 queries, blocking I/O,
   missing caches
114. Style – naming consistency, dead code, overly complex expressions,
   missing imports
125. Robustness – error handling, timeout guards, resource cleanup, input
   validation
13
14 Output your review as a list of findings. Each finding should be a single line
   prefixed with one of:
15 - `[BUG]` for correctness issues
16 - `[SEC]` for security issues
17 - `[PERF]` for performance issues
18 - `[STYLE]` for style issues
19 - `[OK]` if nothing found in a category
```

8. 第八章 Context Compact

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

上下文不是越多越好，而是要把“仍然有用的部分”留在活跃工作面里。

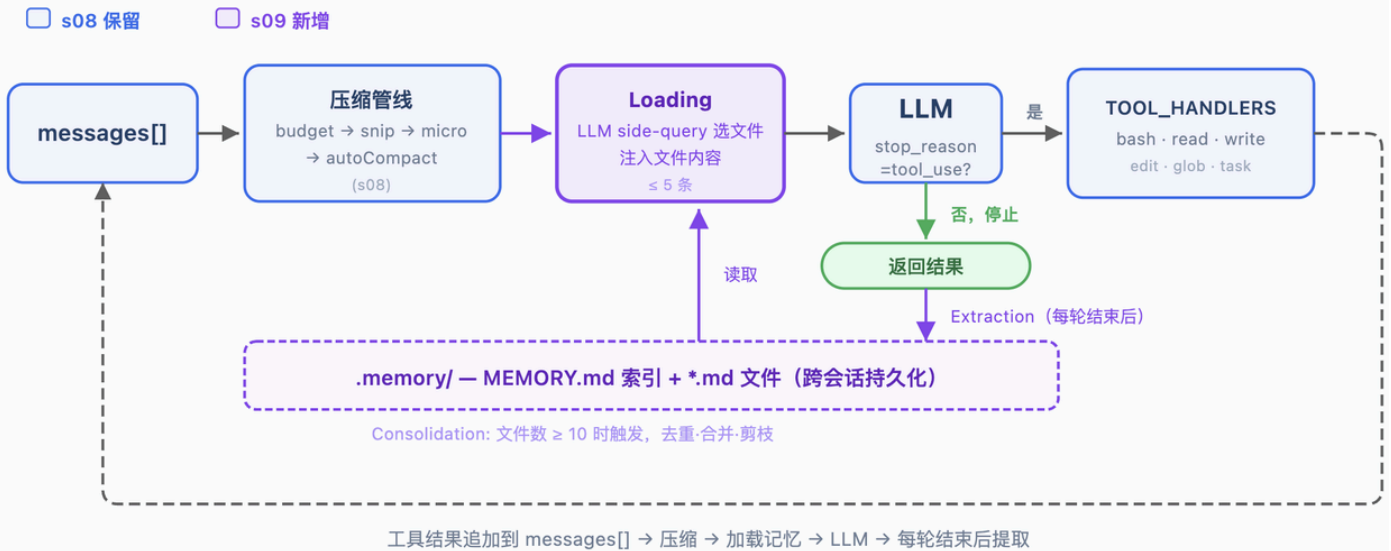
涉及以下问题时可以压缩：

1. 一次会话里面多次调用工具，可以设计一个次数阈值 A来完成工具调用的压缩，只保留最近 3 个工具结果的完整内容，更旧的改成占位显示
2. 当当前上下文 token 测算超过一定阈值 B时，可以对整个会话生成摘要
3. 当某一次工具调用大于设定的长度阈值 C，可以对这次的工具调用结果进行压缩，持久化到本地文件

9. 第九章 Memory

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

Memory — 在 s08 压缩管线上，插入记忆加载、提取与整理



- s08 保留: 压缩管线 (budget → snip → micro → auto) + 应急裁剪 + 循环
- s09 新增: Loading (索引常驻 + 按需注入) + Extraction (每轮结束后) + Consolidation (阈值触发)

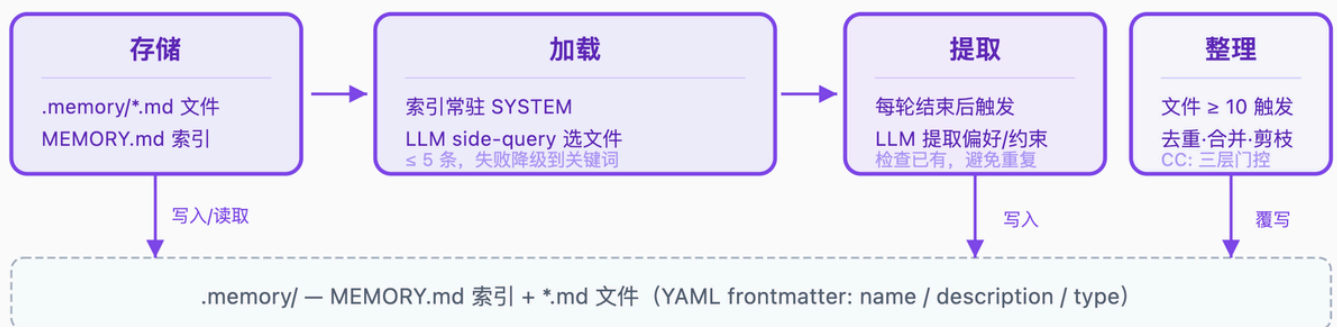
9.1 概述

压缩会把当前目标、剩余工作、用户约束写进摘要，但细节会丢失："用 tab 缩进不要用空格"可能被简化成"用户有代码风格偏好"。而且新开一个会话，连摘要也没了。

LLM 没有持久状态，所有信息都在上下文窗口里。上下文满了要压缩，压缩就有损。需要一层不参与压缩、跨会话保留的存储。

9.2 设计决策

Memory System — 存储 · 加载 · 提取 · 整理



四类记忆: user (你是谁) · feedback (怎么做事) · project (正在发生什么) · reference (东西在哪找)

CC 源码对照

- 记忆选择: LLM side-query (Sonnet 选), 不是 embedding 向量相似度
- 提取时机: stop hook 中触发 (每轮结束后), 不是 autoCompact 后
- Dream 整理: 三层门控 (时间 ≥ 24h + 会话 ≥ 5 + 文件锁), 不是简单计数

1. 小索引让记忆保持低成本

MEMORY.md 保持短小并始终存在。只有当索引显示相关时，才加载完整记忆文件。

2. 记忆提取发生在回合之后

Agent 先完成用户任务，再从原始消息中提取持久事实。这样不会用记忆账务打断工作循环。

3. 整合是周期性的，不是每回合都做

Dream 式整合可以周期性合并和修剪记忆。运行时把日常检索和更重的记忆维护分开。

4. 记忆的存储方案，以本地的 md 文件存储

存储选文件系统：.memory/ 目录下，每个记忆一个 .md 文件，带 YAML frontmatter (name / description / type)。

文件多了需要索引：MEMORY.md 一行一个链接，注入 SYSTEM。

5. 记忆的选择与加载

路径一：索引常驻 SYSTEM。 build_system() 在每次用户请求开始时读取 MEMORY.md，把记忆清单注入。记忆提取和整理只在本轮结束时触发，因此同一轮用户请求中不需要重复重建 SYSTEM。

路径二：相关记忆按需注入。 每次用户请求开始时，load_memories() 把最近对话和记忆目录 (name + description) 一起发给 LLM 做一次轻量 side-query，选出相关的文件名，再读文件内容临时注入到当前 user turn。最多 5 条，控制开销。如果 side-query 失败 (API 错误、JSON 解析失败)，降级到关键词匹配 name + description。

6. 记忆的写入时机

在每轮结束时运行，条件是模型停止且没有 tool_use (说明对话告一段落)。提取前先检查已有记忆，避免重复。提取 prompt 要求 LLM 返回 {name, type, description, body} 的 JSON 数组，只有确实有新信息时才写文件。

7. 做梦

记忆文件会积累。在文件数达到阈值 (默认 10) 时触发，让 LLM 去重、合并矛盾、淘汰过时记忆。

不是"空闲时触发"或"数量够了就合并"，而是四层门控：

- 时间门控：**距上次合并 ≥ 24 小时
- 扫描节流：**避免频繁扫描文件系统
- 会话门控：**自上次合并以来修改了 ≥ 5 个会话 transcript
- 锁门控：**没有其他进程正在合并

9.3 记忆类型

四类记忆，各有用途：

类型	回答什么	示例
user	你是谁	"用 tab 不用空格"
feedback	怎么做事	"别 mock 数据库"
project	正在发生什么	"auth 重写是合规驱动"
reference	东西在哪找	"pipeline bug 在 Linear INGEST"

9.4 工程化落地的思考

9.4.1 记忆的增删改查

上述的Claude Code中的记忆实现在记忆的写入、查询和更新上都是通过代码来强行触发的。还有一种方案是把记忆相关的能力变成Skill或者MCP，完全由模型自主判断调用。

这两种思路并不是非此即彼的，我们通常采用**分层混合架构**。

- 记忆写入：采用在模型运行结束后，手动编码来触发异步任务，由LLM决定是否要记忆、记忆什么内容。
- 记忆查询：1) 索引记忆或者元记忆，在构造system prompt的时候就注入好。2) 其他的记忆由模型通过Skill/MCP根据当前意图灵活决定是否需要查记忆，提高相关性，必要时可以降级使用关键词或向量召回。
- 记忆更新：1) 常规记忆同记忆写入一样，由会话结束后的异步任务执行。2) 复杂记忆（AutoDream）由后台定期总结更新。

9.4.2 记忆的类型



记忆的类型有千千万万，但是最核心应该是去解决以下的问题：

- 进化（Agent 越用越聪明、越个性化）

具体可以从以下几个角度展开：

- 避免重复（避免重复询问用户，保存用户偏好、事实）
- 形成经验（把可复用的经验、规则、SOP和反思沉淀下来）
- 记忆网络（把聊天A—用户A-用户B-聊天B构造成图）

对于不同场景下的Agent有不同的记忆类型设计。

如Claude Code是一个编程助手，它更多的是从个人、项目、团队的角度，设计了user、feedback、project、reference几种记忆类型。

如GenericAgent是一个个人助理，它更多的是从进化、效率的角度，提出了分层的四层记忆架构，L1 索引层、L2 事实层、L3 技能层和L4 归档层。

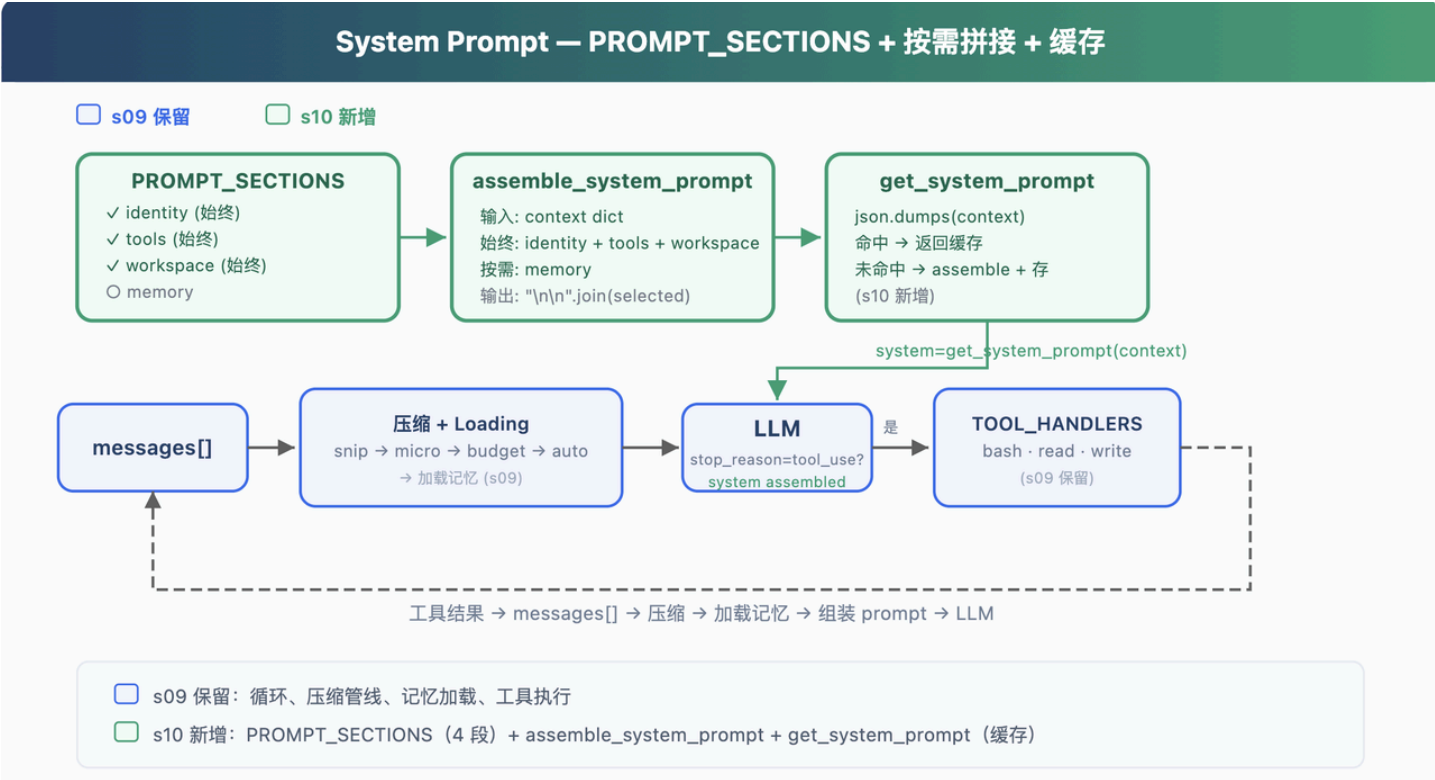
以上两种Agent的记忆设计只满足了避免重复、形成经验，但是由于是使用用户是单体的，并没有构建出记忆网络。

在更宽泛的生产场景中（如客服场景），一般是多用户，我认为可以从用户个人和群体两个维度来设计记忆，再结合用户的元信息构造出图网络。

对于Agent，我认为最终最有竞争力的不会是谁的模型更强，而是谁的 Agent 能够随着服务每一个用户，持续沉淀经验，并把这些经验转化为可复用、可迁移、可推理的知识网络。那时，Memory 就不再是一个存储模块，而是整个 Agent 系统不断成长的核心引擎。

10. 第十章 System Prompt

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>



System prompt 应该是运行时根据当前状态组装的配置：哪些工具启用、哪些上下文可见、哪些记忆相关、哪些内容必须保持稳定以命中 prompt cache。

把硬编码的 `SYSTEM` 拆成独立段落，运行时根据真实状态按需拼接，缓存结果避免重复组装。四个 section，两种加载策略：

Section	加载策略	内容	判断依据
identity	始终	你是谁、怎么做事	始终存在
tools	始终	可用工具列表	enabled_tools
workspace	始终	工作目录	始终存在
memory	按需	相关记忆内容	.memory/MEMORY.md 是否存在

关键设计：section 是否加载取决于真实状态（工具是否存在、文件是否存在），不是消息里的关键词。

CC 的 system prompt 有多少 section?

数量不固定，受 feature flag、output style、KAيروس/Proactive 模式、用户类型、token 预算等影响。大致分两类：

- **静态 section**（始终加载）：identity、system、doing_tasks、actions、using_tools、tone_style、output_efficiency 等。
- **动态 section**（按状态加载）：session_guidance、memory、ant_model_override、env_info_simple、language、output_style、mcp_instructions、scratchpad、frc、summarize_tool_results、numeric_length_anchors、token_budget、brief 等。

11. 第十一章 Error Recovery

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

Error Recovery — try/except 包裹 LLM 调用，三种恢复模式

□ s10 保留

□ s11 新增



错误恢复（分类处理，恢复后回到 LLM 重试）

路径 1 max_tokens 输出被截断 → 升级 8K→64K（一次） / 续写提示（最多 3 次）
触发: stop_reason == "max_tokens" · 代价: 0~1 API · 恢复后 continue

路径 2 prompt_too_long 上下文超限 → reactive compact → 重试（一次机会）
触发: API 返回 413 · 代价: 1 API · 压缩过还是超 → 退出

路径 3 429/529 临时故障 → 指数退避 + 抖动（最多 10 次） / 3 次 529 → 切换模型
触发: RateLimitError / OverloadedError · 公式: $\min(500 \times 2^n, 32s) + \text{jitter}$

三种最常见的恢复模式。CC 实际有 13+ reason code (image_error、aborted_streaming 等)，各有专门处理。
所有路径恢复后 → continue 回到 LLM · 正常流程: 工具结果 → messages → 循环

三种最常见的故障模式：

1. 输出被截断（模型话说一半 token 用完了）
2. 上下文超限（压缩后还是太长）
3. 临时故障（429 限流 / 529 过载）

12. 第十二章 MCP Tools

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

太常见，略。